

PRD: BitGo Agent Wallet

Product: BitGo Agent Wallet (new product line, layered on existing BitGo custody infrastructure)

Status: Draft v1.0

Date: September 2026

Author: [Product Owner]

Positioning Statement

BitGo Agent Wallet is the best-in-class agentic wallet for regulated institutions — not the best agentic wallet in the market, period.

This product is deliberately not competing to be the fastest, most autonomous, or most consumer-accessible agentic wallet. It is competing to be the one an institution can deploy without a compliance or security exception.

- **We will win on:** institutional custody rigor combined with genuine agent-native tooling, audit/compliance depth, and governance-conservative defaults (deny-on-timeout, Strict Mode as default, human approval built in from v1 rather than bolted on).
- **We will not win on, and should not attempt to win on:** raw autonomy/speed (MetaMask Beast Mode, OKX risk-grading), consumer/prosumer accessibility (out of scope by design), ecosystem maturity at launch (competitors have months of live production data and broader chain/framework coverage), or breadth of agent framework integrations at v1 (one flagship integration at launch vs. competitors' five-plus).
- **Why this trade-off is intentional:** chasing autonomy or consumer UX parity would dilute the custody-rigor advantage that is BitGo's actual differentiator, and would ask BitGo to compete on ground where MetaMask, Coinbase, and OKX already have a structural head start (consumer distribution, live threat-detection data at scale, faster iteration cycles).
- **Success looks like:** institutions that need an agent wallet but cannot accept MetaMask/OKX-level autonomy defaults choose BitGo specifically because of this trade-off — not despite it.

Any roadmap decision that would trade governance rigor for speed or autonomy parity with consumer-first competitors should be treated as a positioning risk and escalated before being shipped.

1. Background & Problem Statement

BitGo is a leading institutional custody provider (MPC, multi-sig, policy-governed wallets) but does not currently offer a dedicated **agentic wallet** product. Competitors — MetaMask (Agent Wallet), Coinbase (AgentKit / Agentic Wallet CLI & MCP), Cobo (Agentic Wallet / Pact model), and OKX (Agentic Wallet / Onchain OS) — have all shipped products that give AI agents a scoped, semi-autonomous ability to hold and move crypto assets under programmable rules.

Today, BitGo's approach to "agentic" activity is to apply existing wallet-level policy controls (spend limits, destination allowlists, approval workflows) to agent-initiated transactions. This is secure and institutionally credible, but it is not a differentiated agent-native product — it is general custody policy retrofitted onto a new use case.

Risk of inaction: BitGo's institutional clients are increasingly experimenting with AI agents for treasury operations, trading, reconciliation, and payments. If they can't do this natively within BitGo, they will build agent workflows on top of competitor infrastructure (Coinbase, Cobo) and treat BitGo as a downstream settlement layer only — eroding BitGo's position as the primary wallet infrastructure for institutional clients.

Opportunity: BitGo can own the "institutional-grade agentic wallet" category — nobody currently combines regulator-friendly custody with genuine agent-native tooling (dedicated agent identities, MCP/CLI access, transaction simulation, tiered autonomy). MetaMask and OKX are consumer-first; Coinbase is developer-first but less custody-rigorous; Cobo is enterprise but not custody-audited to BitGo's standard. There's a clear white space for "the agentic wallet institutions can actually pass an audit with."

2. Goals & Success Metrics

Goals

1. Ship a dedicated agent-facing wallet surface that sits alongside (not instead of) BitGo's core custody product.
2. Preserve BitGo's institutional trust position — no reduction in custody rigor, auditability, or governance control.
3. Close the feature gap with MetaMask/Coinbase/Cobo/OKX on agent-native functionality within two to three release cycles.
4. Make it possible for a developer to go from zero to a working agent integration in under a day (parity with Coinbase's AgentKit onboarding).

Success Metrics (first 2 quarters post-launch)

- Number of institutional clients activating at least one agent sub-wallet
- Number of agent-initiated transactions processed

- % of agent transactions auto-approved within policy vs. escalated to human review
- Developer time-to-first-transaction (target: under 4 hours using SDK/CLI/MCP)
- Zero critical security incidents involving agent-initiated transactions
- Net new institutional accounts citing "agent wallet support" as a reason for choosing BitGo (sales-attributed)

3. Target Users

Persona	Description	Primary Need
Institutional treasury team	Manages corporate/fund crypto holdings, wants agents to handle repetitive tasks (rebalancing, reporting, reconciliation)	Governance, auditability, narrow agent scope
Platform/fintech developer	Builds products on top of BitGo custody, wants to embed agent capability	SDK/CLI/MCP access, fast integration, clear docs
Compliance/risk officer	Approves and audits what agents are allowed to do	Full audit trail, policy enforcement, kill switch
Trading desk / quant team	Wants agents executing time-sensitive strategies across sub-wallets	Low-latency execution, parallel sub-wallets, risk-graded autonomy

4. Competitive Gap Summary

Capability	MetaMask	Coinbase	Cobo	OKX
Dedicated agent wallet/identity	✓	✓ (CLI/MCP)	✓	✓
Pre-execution transaction simulation	✓	Partial (KYT)	Partial (policy check)	✓
Threat/malicious contract scanning	✓ (Blockaid)	✓ (KYT)	Partial	✓

Capability	MetaMask	Coinbase	Cobo	OKX
Tiered autonomy modes	✓ (Guard/Beast)	Limited	✓ (Pact retry)	✓ (risk grading)
Multi-channel human approval	Push/email	UI-based	Web/mobile/Telegram/Discord	Automat grading
Agent-specific emergency stop	—	—	✓	✓ (revocati
Native x402 support	✓ (declared)	✓	—	✓
Developer SDK/CLI/MCP for wallet ops	Partial	✓	Partial	✓
Framework integrations published	✓	✓	—	—
Institutional custody pedigree	Low	Medium	Medium	Low-Medium

BitGo's wedge: be the only product that's simultaneously agent-native AND institution-grade. This PRD is scoped to close the ✗ rows without weakening the ✓ BitGo already owns.

5. Scope

5.1 In Scope (v1 / MVP)

Chain scope note: BitGo Agent Wallet v1 targets EVM-compatible chains only (Ethereum mainnet and 7702-supporting L2s). This is a deliberate scope decision, not an oversight — gas sponsorship (Section 6.10) and several agent-native features described below depend on EIP-7702, which is an Ethereum-specific standard. Non-EVM chains (Bitcoin, Solana, etc.) are explicitly out of scope for this product line in v1; supporting them would require a separate technical approach and is not assumed to be a fast-follow.

1. **Agent Sub-Wallet** — a distinct wallet type scoped to a single agent identity, with its own keys/session, nested under an institutional master account.

2. **Policy & Pact Engine** — task-scoped rules (spend caps, protocol/contract allowlists, network restrictions, time-bound sessions) attached to each agent sub-wallet.
3. **Pre-execution Simulation & Screening** — every agent-initiated transaction is simulated (balance/state change preview) and screened against a threat/sanctions list before signing.
4. **Tiered Autonomy Modes** — at minimum two modes: "Strict" (every transaction requires human approval) and "Bounded Auto" (auto-executes within policy, escalates exceptions).
5. **Multi-Channel Approval** — approvals deliverable via BitGo web console, mobile push, and at least one chat integration (Slack, given BitGo's institutional client base — Telegram/Discord as fast-follow).
6. **Agent Kill Switch** — one-click suspension of a single agent sub-wallet's signing ability, separate from freezing the master wallet.
7. **Developer SDK, CLI, and MCP Server (transactional)** — extend the existing docs-only MCP server to support authenticate, send, trade, and balance/status queries.
8. **Audit Log** — immutable, exportable log of every agent action, policy decision, and approval/denial, meeting existing BitGo compliance/export standards.

5.2 Fast-Follow (v1.1–v1.2)

- Native x402 support for agent-to-API micropayments
- Telegram/Discord approval channels
- Published framework integration guides (LangChain, MCP-based agent frameworks, Claude Code, etc.)
- Risk-grading engine (auto-classify transactions by risk tier, similar to OKX)
- Up to N parallel sub-wallets per agent for multi-strategy execution

5.3 Out of Scope (v1)

- Consumer-facing (non-institutional) agent wallet product
- Fully autonomous execution with no policy ceiling (BitGo will not compete on "least human oversight")
- Custodial mode changes to BitGo's core MPC/multi-sig architecture

6. Functional Requirements

6.1 Agent Sub-Wallet Creation

- Institutional admin can create an "Agent Sub-Wallet" nested under a master account via console or API.

- Each agent sub-wallet has: a unique identity/key scope, an assigned Pact (policy set), a funding source (allocated balance or draw-down limit from master wallet), and an autonomy mode.
- Admin can create multiple agent sub-wallets per master account (limit configurable; recommend starting cap of 25, matching enterprise scale needs without over-indexing on OKX's 50).

6.2 Policy / Pact Engine

- Policy fields: max transaction value, daily/weekly spend cap, contract/protocol allowlist, network allowlist, destination address allowlist/denylist, session expiry.
- On a denied action, the system returns a structured denial reason to the agent (machine-readable) so the agent can retry within scope — matching Cobo's retry-feedback pattern.
- Policies must be editable only by users with admin/compliance role; agents cannot self-modify policy.

6.3 Pre-Execution Checks

- Every transaction is simulated prior to signature: shows resulting balance changes, contract interactions, and estimated fees.
- Screening against sanctions lists and known malicious contract/address databases (build or license — evaluate Blockaid, Chainalysis, or existing BitGo compliance vendor).
- Failed screening = hard block, not just a flag.

6.4 Autonomy Modes

- **Strict Mode:** every transaction pauses for human approval regardless of policy compliance.
- **Bounded Auto Mode:** transactions within policy execute automatically; only out-of-policy or flagged transactions escalate.
- Mode is set per agent sub-wallet, changeable only by admin/compliance role, with all mode changes logged.

6.5 Human Approval Flow

- Approval requests deliverable to: BitGo web console, mobile push (existing BitGo mobile app), Slack (v1), Telegram/Discord (fast-follow).
- Configurable approval timeout with default action (default: auto-deny on timeout, not auto-approve — this is a deliberate conservative default distinguishing BitGo from consumer wallets).
- Multi-approver support for high-value transactions (consistent with existing BitGo multi-sig governance model).

6.6 Emergency Stop

- Admin/compliance role can suspend a single agent sub-wallet instantly (revokes signing ability, does not affect other sub-wallets or the master wallet).
- Suspension event triggers immediate notification to all admins on the account.

6.7 Developer Tooling

- **SDK:** language bindings matching BitGo's existing SDK support (at minimum TypeScript/Python) with methods for agent sub-wallet creation, transaction submission, status polling.
- **CLI:** authenticate, create-agent-wallet, send, get-balance, get-status, revoke commands.
- **MCP Server (transactional):** expand current docs-only MCP server to expose the same operations as the CLI, so agent frameworks (Claude Code, LangChain-based agents, etc.) can call BitGo directly.
- Full API reference and quickstart targeting under-4-hour time-to-first-transaction.

6.8 Audit & Compliance

- Every agent action (attempted and executed), policy decision, approval/denial, and mode change is logged immutably.
- Logs exportable in formats compatible with existing BitGo compliance reporting (CSV, API, SIEM integration if already supported).
- Retention period matches BitGo's existing institutional data retention policy.

6.9 Incoming Transaction Screening

- Unlike outgoing transactions, incoming transfers to an agent sub-wallet cannot be pre-screened — screening occurs immediately after on-chain confirmation.
- Every incoming transaction is automatically checked against sanctions lists and known-malicious-source databases (same vendor/engine used for outgoing screening, per Section 6.3).
- Funds identified as flagged (sanctioned origin, known illicit source, mixer-linked, etc.) are tagged and quarantined: excluded from the sub-wallet's spendable/available balance until reviewed, even though they remain visible in the wallet.
- A flagged incoming transaction triggers immediate notification to the account's compliance/admin role, with the same audit-log treatment as any other policy event (Section 6.8).
- Quarantined funds require explicit compliance/admin release before becoming spendable or transferable; an agent cannot self-authorize release of quarantined funds regardless of autonomy mode.

- **Screening latency target:** flag determination within seconds of confirmation, not batched — so an agent's downstream logic (e.g., "spend incoming funds immediately") doesn't act on tainted funds before a flag lands.

6.10 Gas Sponsorship via EIP-7702

- **Scope:** applies only to EVM-compatible chains that support EIP-7702 (Ethereum mainnet and 7702-enabled L2s), consistent with the product's EVM-only chain scope (Section 5.1). Not available on non-EVM networks.
- **Mechanism:** each agent sub-wallet's EOA signs a delegation to an audited delegator/paymaster contract (following the pattern published by MetaMask and Safe). Once delegated, BitGo's paymaster infrastructure covers gas costs for that sub-wallet's transactions, so the agent never needs to hold or manage native gas tokens.
- **Policy integration:** gas sponsorship is not unconditional. Each Pact (Section 6.2) includes a sponsorship cap — a per-transaction and/or daily/weekly gas budget BitGo will cover for that sub-wallet. Transactions exceeding the cap either fall back to the sub-wallet's own funded balance (if available) or are blocked, per policy configuration.
- **Interaction with pre-execution checks:** sponsorship does not bypass simulation or screening (Section 6.3). A transaction is simulated and screened first; sponsorship only determines who pays the resulting gas cost, not whether the transaction is permitted.
- **Dependency:** requires an audited delegation contract before launch. BitGo should not ship a custom, unaudited delegator — evaluate adopting/adapting an existing audited implementation (e.g., patterns published by MetaMask or Safe) rather than building one from scratch for v1.
- **Institutional precedent:** this mirrors the MPC + EIP-7702 combination already publicized by Fireblocks — pairing institutional-grade MPC custody with smart-EOA capabilities like sponsorship, batching, and session keys without giving up custody rigor.

7. Non-Functional Requirements

- **Security:** No reduction in key security model — agent sub-wallet keys must meet the same MPC/HSM standards as existing BitGo wallets. Agent session keys should be scoped and time-bound, never full custody keys.
- **Latency:** Simulation + screening should add no more than ~1–2 seconds to transaction submission for non-flagged transactions, to remain viable for trading-desk use cases.
- **Availability:** Agent wallet infrastructure inherits BitGo's existing SLA commitments for institutional clients.

- **Auditability:** All new functionality must be compatible with BitGo's existing SOC 2 / compliance certifications — no new uncertified data paths.
 - **Backward compatibility:** Must not require changes to existing (non-agent) wallet behavior or client integrations.
-

8. Risks & Mitigations

Risk	Mitigation
Diluting BitGo's institutional/security reputation by moving toward "autonomous" execution	Default to conservative autonomy (Strict Mode default on new sub-wallets); market as "governed autonomy," not "full autonomy"
Screening/threat-detection vendor gap (BitGo has less consumer-facing threat data than Blockaid)	Evaluate build vs. license decision early; consider partnership rather than in-house build for v1
Institutional clients slow to adopt due to compliance review cycles	Ship audit log and compliance export features at v1, not fast-follow, since this is BitGo's differentiator
Feature parity race with faster-moving consumer wallets (MetaMask, OKX)	Do not attempt to match consumer feature velocity; compete on custody rigor + governance instead
Agent framework compatibility lagging competitors	Prioritize MCP server + one flagship integration (e.g., Claude Code) for launch credibility, expand after

9. Rollout Plan

1. **Phase 0 (internal alpha):** Agent sub-wallet + Strict Mode only, internal dogfooding with BitGo's own operational workflows.
 2. **Phase 1 (design partner beta):** 3–5 institutional clients, Strict + Bounded Auto modes, SDK/CLI access, Slack approvals.
 3. **Phase 2 (GA):** Full v1 scope, published docs, MCP server public, marketing positioning as "the institutional agentic wallet."
 4. **Phase 3 (fast-follow):** x402, additional approval channels, risk-grading, framework integration guides.
-

10. Technical Standards Adoption

This section evaluates the EIP/ERC standards relevant to agentic wallet infrastructure and specifies which are adopted for BitGo Agent Wallet, how, and where each maps into the functional requirements in Section 6. All of these are EVM-specific, consistent with the product's EVM-only chain scope (Section 5.1).

Priority key: *Required* = the product does not work as designed without it, no substitute. *Recommended* = a security/interoperability upgrade over a substitutable backend-only approach; skipping it still "works" but reopens a risk or loses interoperability the PRD otherwise commits to. *Conditional* = only required if a specific other design choice is made (noted inline). *Optional* = UX/efficiency polish; the wallet functions fully without it.

10.1 Adopt Now (v1 Foundation)

Standard	Priority	What it does	How BitGo Agent Wallet uses it
EIP-7702	Required	EOA temporarily delegates to smart-contract code	Base layer for agent sub-wallet delegation and gas sponsorship (Section 6.10); already scoped as the foundation of the EVM-only chain decision (Section 5.1)
ERC-4337	Required	UserOperations, bundlers, paymasters	Execution backbone underneath ERC-7579/7710/7715/7902; required for gas sponsorship (Section 6.10) to work at production scale and for transaction batching (Section 6.7)
ERC-7710	Recommended	Onchain Delegation Manager that enforces scoped permissions	Implement the Pact engine (Section 6.2) as a 7710-compliant Delegation Manager contract, so policy limits are enforced onchain at signing time — not only in BitGo's backend. Substitutable with backend-only

Standard	Priority	What it does	How BitGo Agent Wallet uses it
			enforcement, but that reopens a bypass risk if the backend is compromised
ERC-7715	Recommended	<code>wallet_grantPermissions</code> — standard request layer for scoped, time-bound permissions	Expose Pact creation (Section 6.2) via the standard RPC so any 7715-aware agent framework (Claude Code, LangChain-based agents, etc.) can request a scoped permission from BitGo without a custom integration. Substitutable with a proprietary API, at the cost of losing out-of-the-box framework interoperability
ERC-7579	Recommended	Modular smart account (validators, executors, hooks, fallback handlers)	Build agent sub-wallets (Section 6.1) as 7579-compliant accounts; implement pre-execution simulation/screening (Section 6.3) as an installable hook module and audit logging (Section 6.8) as an executor module. Substitutable with a monolithic custom contract, at the cost of harder auditing and no independent module upgrades
EIP-1271	Required	<code>isValidSignature</code> interface for contract wallets	Required specifically because BitGo's custody model is MPC/multi-sig, not a single ECDSA key —

Standard	Priority	What it does	How BitGo Agent Wallet uses it
			without it, agent sub-wallets cannot validate their own signatures to external protocols
EIP-712	Required	Typed, structured data signing (vs. blind-signing raw hashes)	Mandatory format for every human approval request (Section 6.5) — approvers see readable typed fields (amount, destination, contract) instead of opaque hex. Required to keep the no-blind-signing commitment already made in Section 6.5

10.2 Fast-Follow (v1.1–v1.2)

Standard	Priority	What it does	How BitGo Agent Wallet uses it
ERC-5792	Optional	<code>wallet_sendCalls</code> / <code>wallet_getCapabilities</code>	Lets agents batch multiple operations into a single call and lets a sub-wallet advertise supported capabilities — extends the SDK/CLI/MCP surface (Section 6.7). UX/efficiency polish, not required for core function
ERC-2612 (Permit)	Optional	Gasless ERC-20 approval via signature	Lets agents approve token spend without a separate on-

Standard	Priority	What it does	How BitGo Agent Wallet uses it
			chain approve() transaction. Only helps for tokens that implement Permit; the wallet functions fully without it
ERC-3009 (transferWithAuthorization)	Conditional	Signature-based token transfer (used by USDC)	Directly supports the x402 fast-follow (Section 5.2) for stablecoin- denominated agent payments. Required only if the x402 payment feature ships — deprioritizing x402 removes the need for this
ERC-6492	Conditional	Signature validation for undeployed (counterfactual) contract wallets	Needed only if agent sub- wallets are deployed counterfactually (address computed before on-chain deployment, to save setup gas). Not needed if sub- wallets are deployed eagerly at creation time

Standard	Priority	What it does	How BitGo Agent Wallet uses it
ERC-7902	Optional	Wallet Capabilities for Account Abstraction	Defense-in-depth tightening of UserOperation validity windows and paymaster parameters, reinforcing Pact session expiry (Section 6.2) at the protocol level. A good-to-have hardening layer, not a blocker

10.3 Watch / Evaluate Later (not v1, needs dedicated review)

- **ERC-7683 (cross-chain intents)**: powerful for expressing cross-chain agent actions ("rebalance to X") as an intent fulfilled by a solver network, but introduces a new trust surface — the solver — not covered by Section 6.3's screening model. Requires its own risk assessment (solver vetting, fallback-if-unfilled behavior) before being added to the roadmap.
- **ERC-7682 (Auxiliary Funds Capability)**: relevant if the sub-wallet funding model (Section 6.1) allows draw-down from a pooled master-account balance exceeding the sub-wallet's own on-chain balance; lets external dApps recognize real spending power instead of incorrectly blocking the agent. Low urgency — revisit once the funding model is finalized.
- **ERC-7802 (cross-chain token mint/burn interface)**: only relevant if BitGo issues its own token or stablecoin requiring standardized cross-chain interoperability. No near-term need identified; track for awareness only.

10.4 Explicitly Rejected

- **ERC-6900**: a competing modular smart-account standard to ERC-7579 (Alchemy-led vs. the broader Safe/Biconomy/ZeroDev/OKX consortium behind 7579). Adopting both would fragment the module security surface for no functional benefit — ERC-7579 has broader vendor support and is the better fit for this project.
- **ERC-2771**: the meta-transaction trusted-forwarder pattern is functionally superseded, for this use case, by the ERC-4337 + EIP-7702 gas-abstraction path

already adopted. Maintaining a second relay pattern adds attack surface without added capability.

10.5 Mapping to Functional Requirements

Functional requirement	Implemented via
6.2 Policy / Pact Engine	ERC-7710 (onchain enforcement), ERC-7715 (request layer), ERC-7902 (session/time-bound scoping)
6.3 Pre-Execution Checks	ERC-7579 (hook module for screening/simulation)
6.5 Human Approval Flow	EIP-712 (typed, human-readable approval payloads)
6.7 Developer Tooling	ERC-5792 (batching/capability discovery), ERC-7715 (standard permission-request RPC for SDK/CLI/MCP)
6.10 Gas Sponsorship	EIP-7702 (delegation), ERC-4337 (paymaster/bundler infrastructure), ERC-2612 / ERC-3009 (fast-follow, gasless token approvals/transfers)

11. Open Questions

- Build vs. license for threat/contract screening (Blockaid-equivalent)?
- Which chat platforms matter most to BitGo's institutional base — is Slack sufficient for v1, or is Telegram non-negotiable given crypto-native client habits?
- Should agent sub-wallet limits (cap per master account) be tied to account tier/pricing?
- Does x402 support need legal/compliance review before fast-follow, given BitGo's institutional/regulatory posture?
- What's the pricing model for agent sub-wallets — per-wallet fee, transaction volume, or bundled into existing custody tiers?

12. Intelligence Layer (Post-v2)

The functional requirements in Section 6 are governance-first: fixed policy limits, deterministic checks, human approval. This section captures AI-native capabilities that go beyond static rule enforcement — genuinely "smart" additions rather than more

policy configuration. None of these are required for v1 or v2; they represent a third horizon once the core governance model is proven in production.

12.1 Behavioral Anomaly Detection

Build a behavioral baseline per agent sub-wallet (typical transaction size, frequency, counterparties, timing) and flag deviations even when a transaction is technically within Pact policy limits (Section 6.2). Catches a compromised or prompt-injected agent acting within its authorized limits but out of character — a gap the current fixed-threshold model doesn't cover.

12.2 Natural-Language Policy Authoring

Let admins describe a Pact in plain English ("let this agent trade up to \$10k/day on Uniswap and Aave, nothing else") and compile it into a formal ERC-7710 delegation, rather than hand-configuring policy fields directly. Lowers setup friction for compliance staff who aren't smart-contract-literate.

12.3 Plain-Language Approval Summaries

Extend the human approval flow (Section 6.5) with a natural-language summary of agent intent alongside the raw EIP-712 typed data ("Agent wants to swap 5,000 USDC for ETH on Uniswap — within its typical pattern"). Faster, better-informed approver decisions without removing the underlying typed-data guarantee.

12.4 Adaptive Autonomy — Agent Trust Scoring

Track a rolling trust score per agent sub-wallet based on denial rate, quarantine events, and policy exceptions over time. An agent with a sustained clean record becomes eligible for a human-approved graduation from Strict Mode to Bounded Auto Mode (Section 6.4), turning autonomy level from a one-time static setting into something that adapts to demonstrated behavior.

12.5 Policy Simulation / Dry-Run Sandbox

Let an institution test a proposed Pact against historical transaction data before going live, to see what it would have approved, denied, or flagged. Surfaces overly loose or overly strict policies before they cause a production incident.

12.6 Cross-Agent Risk Correlation

Section 6.2's Pact enforcement is per sub-wallet only. Add a check across an institution's sub-wallets for emergent risk — e.g., several agents each individually within limits but collectively concentrating exposure in one counterparty or protocol. Becomes necessary once an institution runs more than a handful of agents.

12.7 Predictive Budget Alerts

Notify admins proactively when an agent is trending toward exhausting a spend cap or gas sponsorship budget (Section 6.10), rather than only reacting once the cap is hit.

12.8 Natural-Language Audit Queries

Let compliance query the audit log (Section 6.8) conversationally ("show every transaction this agent attempted that got denied last week") instead of writing queries against structured exports.

12.9 Graded Risk Scoring on Incoming Screening

Extend incoming transaction screening (Section 6.9) from a binary flagged/clean outcome to a graded score with reasoning, so compliance can triage quarantined funds by severity instead of treating every flag identically.

12.10 Multi-Agent Hierarchy / Delegation Chains

Use ERC-7710's attenuation support (Section 10.1) to let a "supervisor" agent hold a broad Pact and delegate narrower sub-Pacts to specialized execution agents — mirroring real treasury org structures (one orchestrator, several narrow-scope workers) instead of requiring every agent to receive its own top-level grant directly from a human.

13. Full Competitive Comparison

This section consolidates every capability covered in this PRD (Sections 6, 10, and 12) into a single comparison against the competitors introduced in Section 4. Where a competitor's public documentation doesn't confirm a capability, it's marked "Not publicized" rather than assumed absent — the goal is an honest picture, not a stacked scorecard.

Capability	MetaMask	Coinbase	Cobo	OKX
Dedicated agent wallet/identity	✓	✓ (CLI/MCP)	✓	✓
Custody model	Self-custodial (TEE)	Non-custodial	MPC, non-custodial default	Self-custodia (TEE)
Chain scope	Multi-chain (9+ EVM chains + Hyperliquid)	EVM + Solana	Multi-network	~20 networks incl. non-EVM-styl Layer

Capability	MetaMask	Coinbase	Cobo	OKX
Pre-execution simulation	✓ (mandatory)	Partial (KYT)	Partial (policy check)	✓ (plain language)
Threat/malicious contract screening	✓ (Blockaid)	✓ (KYT)	Partial	✓
Incoming transaction screening + quarantine	Not publicized	Not publicized	Not publicized	Not publicize
Tiered autonomy modes	✓ (Guard/Beast)	Limited	✓ (Pact retry)	✓ (risk grading)
Onchain policy enforcement (delegation manager)	Not publicized as ERC-7710-based	Not publicized	Proprietary Pact model	Not publicize
Standard permission request layer	Uses ERC-7715 internally (Advanced Permissions)	Not publicized	Proprietary	Not publicize
Mandatory typed-data signing (no blind-signing)	Industry standard (EIP-712)	Industry standard	Industry standard	Industry standard
Gas sponsorship (EIP-7702-based)	Not publicized for Agent Wallet specifically	Not publicized	Not publicized	Not publicize
Multi-channel human approval	Push/email	UI-based	Web/mobile/Telegram/Discord	Automat grading
Agent-specific emergency stop	—	—	✓	✓ (revocati

Capability	MetaMask	Coinbase	Cobo	OKX
Native x402 support	✓ (declared)	✓	—	✓
Developer SDK/CLI/MCP (transactional)	Partial	✓	Partial	✓
Framework integrations published	✓ (5+)	✓	—	—
Behavioral anomaly detection	Not publicized	Not publicized	Not publicized	Not publicize
Natural-language policy authoring	Not publicized	Not publicized	Not publicized	Not publicize
Adaptive trust-based autonomy	Not publicized	Not publicized	Not publicized	Not publicize
Cross-agent risk correlation	Not publicized	Not publicized	Not publicized	Not publicize
Multi-agent delegation hierarchy	Not publicized	Not publicized	Not publicized	Not publicize
Institutional custody pedigree	Low	Medium	Medium	Low-Medium
Compliance audit log depth	Not publicized	Not publicized	Moderate	Not publicize

13.1 What BitGo Target v1 has that no competitor is confirmed to have

- Incoming transaction screening with quarantine (Section 6.9) — every competitor's public materials focus on outgoing screening only.
- Onchain policy enforcement via a standardized delegation manager (ERC-7710) rather than a purely proprietary or backend-only policy model.

- A standardized permission-request layer (ERC-7715) exposed specifically for agent Pact creation, not just general dapp permissions.
- Institutional custody pedigree combined with genuine agent-native tooling — this remains the core wedge from Section 4, now backed by concrete v1 scope rather than just a positioning claim.

13.2 Where BitGo Target v1 still lags at launch

- Ecosystem maturity: MetaMask, Coinbase, and OKX have months of live production data, broader chain coverage, and larger framework integration lists.
- Autonomy/speed: Beast Mode (MetaMask) and automated risk-grading (OKX) offer less friction than BitGo's Strict/Bounded Auto default posture — a deliberate trade-off per the Positioning Statement, not an oversight.
- x402 and native stablecoin payment rails: fast-follow for BitGo, already shipped at Coinbase and OKX.
- Chain breadth: BitGo Target v1 is EVM-only by design (Section 5.1); MetaMask, Coinbase, and OKX all support non-EVM or broader multi-chain footprints today.

13.3 What closes the remaining gap

The Post-v2 Intelligence Layer (Section 12) is where BitGo would move from "caught up" to "ahead" — none of the behavioral, adaptive, or natural-language capabilities in Section 12 are confirmed anywhere in the competitive set. That said, this is forward-looking and unbuilt; it should be read as a roadmap advantage, not a current one.
